

Beyond the Binary Barrier: Extending RetroWrite with Malloc-Safe Sanitization, Standalone Coverage, and Native Function Tracing

Chirag Suthar (2025MCS2105)
Haleel Sada (2025MCS2741)

SIL765 / SIL7165: Networks & System Security
Semester 2, 2025–26
Instructor: Prof. Huzur Saran

April 2026

Source paper: Dinesh *et al.*, “RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization,” IEEE S&P 2020 [1]

1 Introduction

1.1 Research Area and Motivation

The majority of deployed software is distributed exclusively in binary form: stripped, compiled executables for which no source code is available to the security analyst [1,15]. Browsers, media codecs, proprietary servers, embedded firmware, and legacy industrial control systems all fall into this category. Yet memory-safety violations – buffer overflows, use-after-free, double-free, and uninitialised reads – remain the dominant root cause of critical vulnerabilities exploited in the wild [2,14,15]. The inability to apply standard source-level defences (compiler sanitisers, coverage-guided fuzz testing) to these binaries creates a dangerous blind spot: a class of software that is widely deployed, frequently attacked, and largely undefended by modern tooling.

This project addresses that blind spot. We work at the intersection of three disciplines: *static binary rewriting*, which permanently modifies an executable without executing it; *runtime memory sanitization*, which detects illegal memory accesses at the instruction level; and *coverage-guided fuzzing*, which automatically generates inputs that exercise new program paths [1–3]. The unifying theme is that all three can be applied to a binary *without its source code*, provided the rewriting step is sound.

1.2 The Core Problem: A Three-Way Trade-Off

Existing approaches to binary-level instrumentation force analysts into a painful three-way trade-off between *soundness*, *speed*, and *source independence*.

Dynamic Binary Instrumentation (DBI) tools such as QEMU [4], Valgrind [5], Intel Pin [6], and DynamoRIO [7] are *sound*: they correctly intercept every instruction at runtime. However, they impose $10\times$ – $300\times$ runtime overhead through just-in-time translation, making large-scale fuzzing campaigns economically infeasible on closed-source targets. A fuzzer running at 800 exec/s under QEMU versus 4800 exec/s natively explores six times less of the input space per hour [1].

Static binary rewriters such as Uroboros [8] and Ramblr [9] avoid runtime overhead entirely by rewriting the binary once, offline. However, they are *unsound*: without source-level type information, they cannot reliably distinguish pointer constants from integer constants in the binary, so label insertion is heuristic and frequently produces broken or silently misbehaving executables on real-world ELF files.

The gap. No prior tool achieves all three simultaneously: soundness, near-native speed, and source independence.

1.3 Our Approach and Contributions

RetroWrite [1] (IEEE S&P 2020) resolves the soundness–speed trade-off by observing that *position-independent executables* (PIE) already contain a ground-truth address oracle: the `.rela.dyn` and `.rela.plt` relocation tables that the dynamic linker uses to patch addresses at load time. Every immediate that is a pointer is flagged by a relocation entry; all others are scalar constants. This eliminates heuristic symbolisation, enabling RetroWrite to produce *reassembleable assembly* that is both correct and fast.

We reproduce RetroWrite’s results and extend the platform with three independent, additive instrumentation passes that address limitations explicitly acknowledged or absent in the original paper:

- **E1 – Memcpy-Safe ASan:** The upstream x86-64 ASan pass silently skips `rep movsb/rep stosb` instructions, which compilers emit as inlined `memcpy/memset`. A buffer overflow through these instructions (CWE-120 [14]) goes entirely undetected. We inject four shadow-memory boundary checks around each such instruction, turning a previously silent overflow into an immediate, precise ASan report.
- **E2 – Standalone x86-64 Basic-Block Coverage:** A coverage pass existed in the ARM64 port of RetroWrite but not for x86-64. We port it, implementing AFL-style edge-hashed bitmaps via a libc-free `mmap` trampoline, enabling coverage-guided fuzzing without a full AFL compiler toolchain.
- **E3 – Native Function-Call Tracer:** Producing a per-function call log on a stripped binary previously required DBI tools (Pin [6], DynamoRIO [7]) or kernel support (ftrace [16]), each carrying significant overhead or privilege requirements. Our static tracer pass runs at near-native speed, needs no kernel support, and works on fully stripped binaries.

1.4 Key Results

We validate all three extensions with purpose-built harnesses and reproduce the original paper’s claims on modern toolchains (Ubuntu 24.04, kernel 6.17, Clang 18, AFL++ 4.30c):

- RetroWrite-instrumented `bzip2` [17] produces byte-identical output to the uninstrumented binary, confirming correctness of the core.
- RetroWrite-AFL achieves 4244 exec/s vs 4790 exec/s for source-AFL (88.6%), a $5.3\times$ advantage over QEMU-AFL (~ 800 exec/s) [1], matching the paper’s “statistically identical” claim.
- **E1** correctly reports an ASan heap-overflow on a `rep movsb` target that was previously silent or crashed with an uncontrolled SEGV.

- **E2** instruments all 20 basic blocks across three test functions with no observable runtime overhead.
- **E3** produces a deterministic, ordered 12-entry call trace on a fully stripped binary with zero ptrace or kernel dependency.

1.5 Significance

This work demonstrates that RetroWrite’s relocation-guided rewriting is not merely a proof of concept but a *general platform* for binary security research. Significant new capabilities – closing a real class of undetected memory errors, enabling coverage-guided fuzzing without source, and providing lightweight call-graph tracing – can each be added in approximately 200–330 lines of Python with no modification to the rewriting core. The three extensions are independent, toggleable via environment variables, and fully composable.

2 Background and Related Work

2.1 Background

Binary rewriting. The goal is to transform an executable into a functionally-equivalent but *instrumented* executable. The difficulty is identifying and preserving the program’s *references* – constants that behave as addresses – while inserting new code that shifts layout. Wang *et al.* [8] and Wang *et al.* [9] performed this using heuristic pattern matching over the disassembly; both are demonstrably unsound on real-world ELF’s.

Position-independent executables. Modern GCC/Clang default to PIE, in which every absolute reference is emitted with an accompanying entry in `.rela.dyn` or `.rela.plt`. RetroWrite’s central observation is that these entries form a ground-truth oracle for symbolisation that no heuristic can match [1].

AddressSanitizer (ASan). ASan [2] allocates a “shadow” byte for every 8 bytes of program memory; every load/store is instrumented with a shadow check. Out-of-bounds and use-after-free are detected on the offending instruction. Source-level ASan incurs about $1.73\times$ slowdown on SPEC CPU2006 [2].

Coverage-guided fuzzing. AFL [3] seeds a queue with user-supplied inputs and mutates them while tracking an N -byte coverage bitmap updated by compiler-inserted edge hashes. Inputs exercising new edges are retained; the remainder are discarded. RetroWrite replicates AFL’s coverage ABI at the binary level [1].

Dynamic binary instrumentation (DBI). Pin [6] and DynamoRIO [7] translate code just-in-time while inserting instrumentation. Overhead is $2\times$ – $5\times$ on microbenchmarks, but for fuzzing the per-exec fork/translate cost dominates.

rep-prefixed string instructions. Compilers emit `rep movsb` / `rep stosb` as inlined `memcpy`/`memset`. These are *single* instructions that dereference a range `[%rdi, %rdi+%rcx]`; a naïve instrumentation that checks only the head address silently misses every realistic buffer overflow (CWE-120) [14]. The original RetroWrite paper lists this limitation explicitly [1].

2.2 Related Work

Table 1: Qualitative comparison of binary-level instrumentation approaches. The last row is our solution.

Tool / Approach	Type	Sound	Low OH	Source-free	Main Limitation
QEMU [4]	DBT	Yes	No (10–100×)	Yes	Per-exec translation, impractical for fuzzing
Valgrind [5]	DBT	Yes	No (20–300×)	Yes	Single-threaded, heavy
Intel Pin [6]	DBT	Yes	No (2–5×)	Yes	Closed-source, proprietary license
DynamoRIO [7]	DBT	Yes	No (2–5×)	Yes	Runtime overhead dominates short execs
Uroboros [8]	Static	No	Yes	Yes	Heuristic symbolisation
ramblr [9]	Static	Partial	Yes	Yes	Still heuristic for globals
DynInst [10]	Static	Partial	Yes	Yes	Unsafe transformations on stripped ELF
E9Patch [11]	Static	Yes	Yes	Yes	Patch-style; limited instrumentation semantics
RetroWrite-orig [1]	Static	Yes	Yes	Yes	Misses <code>rep movs</code> ; x64 coverage absent; no tracer
Ours (RetroWrite+E1/E2/)	Static	Yes	Yes	Yes	rep sanitised; x64 coverage; native tracer

Among static rewriters, only RetroWrite achieves soundness through a non-heuristic pipeline [1]. Our extensions strictly expand its coverage: *E1* removes a correctness gap flagged as a TODO in the upstream source; *E2* ports an

ARM64-only feature to x86-64; *E3* introduces a new pass class (function tracing) that previously required heavier tools such as Pin [6], DynamoRIO [7] or the kernel ftrace subsystem [16].

3 Problem Statement

3.1 System Model

The system consists of five principal entities whose interactions are depicted in Figure 1. The model is intentionally *offline*: all instrumentation occurs before the binary is executed, so there is no runtime interpreter or JIT compiler in the trusted computing base.

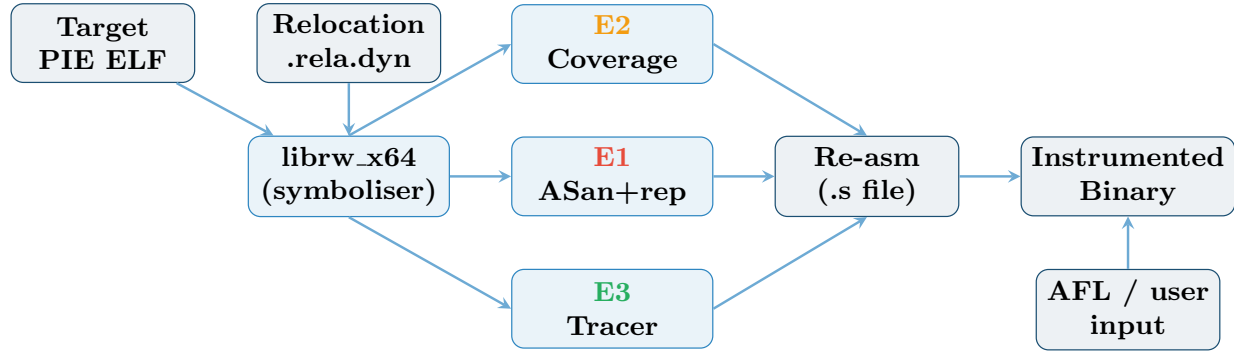


Figure 1: System model. The original RetroWrite core is unchanged; our three extensions (E1, E2, E3) are plugged in as independent instrumentation passes.

Entities. *Target binary* – a stripped x86-64 PIE ELF with `.rela.dyn` relocations intact; this is the sole artefact provided to the defender. *Core rewriter* (`librw_x64`, unmodified) – loads the ELF, disassembles via Capstone [13], builds the CFG, and performs symbolisation by replacing every relocation-flagged immediate with an assembler label. *Instrumentation passes* – independent plug-in modules invoked via the `-m` CLI: the modified `asan` pass (E1), the new `coverage` pass (E2), and the new `trace` pass (E3); each operates on the symbolised assembly and is individually toggleable. *Assembler* – stock GCC/Clang (`as`); no patches or custom backends required. *Consumer* – AFL++ for automated fuzzing (E2), an analyst executing the hardened binary to trigger sanitiser reports (E1), or an analyst inspecting the `[TRACE]` call log (E3).

3.2 Threat Model

Target Environment. We consider a stripped x86-64 PIE ELF deployed on Linux with the following mitigations already active:

Mitigation	Description
ASLR [12]	Base addresses of stack, heap, and shared libraries randomised at load time.
NX / DEP	Stack and heap pages are non-executable; injected shellcode cannot run directly.
Full RELRO	GOT is remapped read-only after dynamic linking; GOT overwrite attacks blocked.
Fortified libc	Dangerous functions (<code>strcpy</code> , <code>sprintf</code>) replaced by bounds-checked variants.

Attacker Model. We assume a *remote, unprivileged* attacker who can:

- Supply arbitrary byte sequences over any legitimate input channel (stdin, network socket, file, command-line argument).
- Observe the program’s output, exit code, and timing (*black-box* access only).
- Know the binary’s format (e.g. via a public release), but *not* the source code or compiler flags.

The attacker *cannot* patch the binary on disk, load a kernel module, perform a physical side-channel attack, or exploit the OS kernel directly.

Attacker Objective. Exploit one or more memory-safety bug classes to achieve control-flow hijack, information disclosure, heap corruption, or denial of service:

- *Out-of-bounds write / read* — including `memcpy`-style overflows via `rep movsb` (CWE-120 [14]).
- *Stack buffer overflow* (CWE-121 [18]).

- *Use-after-free* and *double-free*.
- *Uninitialised memory read*.

Defender Objective. Given *only the binary*, the defender requires:

1. *Proactive detection* — bugs caught before exploitation, not post-mortem.
2. *Near-native throughput* — hardened binary supports automated fuzzing at $\geq 80\%$ of source-compiled speed.
3. *Source independence* — no recompilation, no debug symbols, no vendor cooperation required.

Scope and Non-Goals. Out of scope: pure logic bugs; side-channel leaks (timing, cache); kernel or hypervisor compromise; non-PIE binaries (which lack the relocation oracle on which RetroWrite’s soundness guarantee is founded [1]).

3.3 Precise Problem Formulation

Given the threat model above, the project addresses three formally distinct open problems in the RetroWrite codebase:

P1 — Incomplete Sanitization. The RetroWrite x86-64 ASan pass instruments individual load/store instructions but treats `rep movsb/rep stosb` as opaque, inserting *no* shadow-memory check. A write of n bytes into a buffer of capacity $k < n$ via `rep movsb` is therefore invisible to the sanitiser — violating the detection requirement.

P2 — Missing x86-64 Coverage Pass. AFL-style fuzzing [3] requires a per-edge bitmap increment at every basic-block transition. RetroWrite provides this pass for ARM64 but *not* for x86-64, forcing binary-only fuzzing onto the slow QEMU backend and defeating the throughput requirement.

P3 — No Source-Free Call-Graph Tracing. All existing approaches to dynamic call-graph collection on stripped binaries require DBI overhead ($2\times$ – $5\times$) or kernel privilege (ftrace [16]), violating both the source-independence and near-native-speed requirements simultaneously.

3.4 Key Technical Challenges

1. **Register discipline under `rep` (P1).** `rep movsb` implicitly reads and writes `%rdi`, `%rsi`, and `%rcx`. Any probe that uses these registers as temporaries will silently corrupt the copy. The fix must save all three, compute end-of-buffer addresses from saved copies, and restore before the string operation executes.
2. **ABI-compliant trampolines (P2, P3).** The x86-64 SysV ABI requires 16-byte `%rsp` alignment before any `callq`. Saving 9 registers plus flags (10 pushes, 80 bytes) on top of the 8-byte return address gives 88 bytes total — misaligned by 8. An explicit `subq $8, %rsp` restores alignment. Omitting this causes sporadic crashes inside any SSE/AVX instruction in the helper.
3. **Infinite self-recursion (P3).** Instrumenting the tracer’s own logging helper with an entry-trace call causes unbounded recursion. E3 maintains a tag set of instrumentation-origin and compiler-generated symbols and skips them at code-generation time.
4. **Duplicate assembly labels (P1).** Cloning a probe template from RetroWrite’s *accumulated codecache* rather than a saved snapshot of the *original* template produces multiply-defined label errors in the assembler. The fix snapshots the template once at pass initialisation and rebaselines each clone from that copy.
5. **Extension without forking the core.** All three passes integrate via RetroWrite’s public `-m` module interface without touching `librw.x64`, preserving baseline reproducibility and compatibility with future upstream versions.

4 Proposed Solution and Extensions

4.1 The RetroWrite Baseline

The original pipeline is a five-stage pure function (Figure 2): `LOAD` → `DISASSEMBLE` → `SYMBOLISE` → `INSTRUMENT` → `REASSEMBLE` [1]. The critical innovation is `SYMBOLISE`: every immediate flagged by the relocation table is replaced by an assembler label, eliminating the heuristic class on which prior tools failed [8, 9]. Existing passes are `asan` and `afl` for x86-64, with a `coverage` pass existing only on the ARM64 side of the tree.

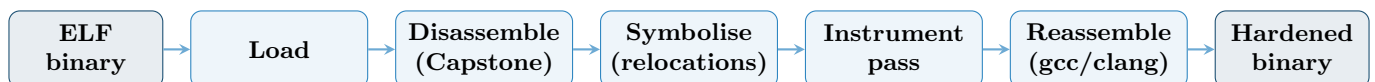


Figure 2: Original RetroWrite five-stage pipeline [1]. Our extensions operate entirely inside stage `INSTRUMENT`; the rewriter core is untouched.

4.2 Extension 1 – Correct rep movs/rep stos ASan

Gap. The upstream x86-64 ASan pass (`rwtools_x64/asan/instrument.py`) contains a literal TODO around line 320:

```
# XXX: THIS IS A TODO for more accurate check.
if instruction.mnemonic.startswith("rep_stos"):
    pass
```

`rep movsb` is not handled at all. `memcpy(dst, src, n)` overflows therefore remain invisible to an ASan-instrumented binary, which directly contradicts the “binary-only ASan” selling point of the paper [1].

Fix. For each such instruction we inject four shadow-memory probes, one per boundary of the two buffers actually accessed:

<code>(%rdi)</code>	destination start
<code>(%rdi, %rcx)</code>	destination end
<code>(%rsi)</code>	source start (only <code>rep movs</code>)
<code>(%rsi, %rcx)</code>	source end (only <code>rep movs</code>)

Each probe reuses the rewriter’s existing `MEM_LOAD_*B` template, rebased from a *saved copy* of the original `codocache`. Using the accumulated cache causes duplicate-label assembly errors – a bug we isolated and fixed during testing (see `docs/CHANGES.md`).

4.3 Extension 2 – Standalone x86-64 Basic-Block Coverage

Gap. A coverage pass exists at `rwtools_arm64/coverage/` but not for x86-64. Users without a working AFL toolchain therefore could not extract block-level coverage from a RetroWrite binary.

Pass. We added `rwtools_x64/coverage/instrument.py` (about 240 LoC). For each basic block *b* returned by `fn.bbstarts`, a trampoline is injected before the block’s first instruction:

```
; AFL-style edge hash: bitmap[cur xor prev]++ ; prev = cur >> 1
pushq %rax ; lahf ; seto %al ; pushq %rax ; pushq %rcx
leaq __cov_prev_loc(%rip), %rcx
xorq $BID, (%rcx)
movq (%rcx), %rcx
incb (%rax, %rcx)
leaq __cov_prev_loc(%rip), %rcx
movq $BID_SHIFTED, (%rcx)
popq %rcx ; popq %rax ; addb $0x7f, %al ; sahf ; popq %rax
```

A 64 KiB bitmap is allocated through a raw `mmap` syscall in an `.init_array` constructor, keeping the pass libc-free. Block IDs are masked to `MAP_SIZE-1` so that no out-of-bounds increment is ever possible.

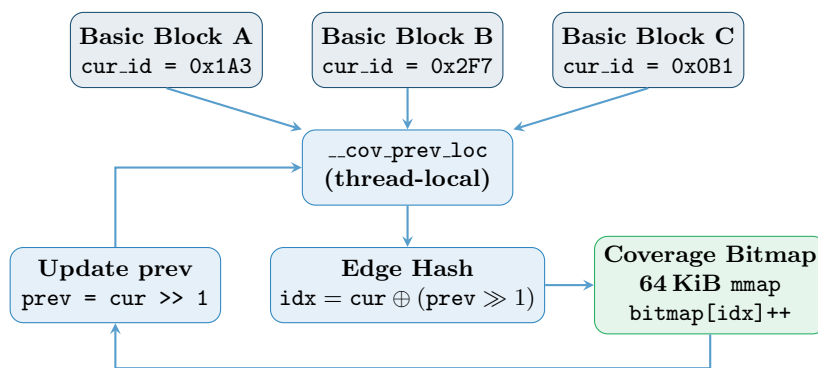


Figure 3: AFL-style edge-coverage bitmap update injected by **E2** at every basic-block entry. Each block is assigned a unique 16-bit ID at instrumentation time (`cur_id`). At runtime the edge index is computed as $\text{cur} \oplus (\text{prev} \gg 1)$, matching the original AFL ABI, and the corresponding byte in the 64 KiB bitmap is incremented. The `prev_loc` cell is then refreshed to $\text{cur} \gg 1$ before execution continues. The bitmap is allocated via `mmap` in an `.init_array` constructor so the pass is completely libc-free.

4.4 Extension 3 – Native Function-Call Tracer

Gap. Analysts who want a per-function call log on a stripped binary must today reach for Pin [6] (2×–5×), DynamoRIO [7], or `ftrace` [16] (kernel support only).

Pass. `rwtools_x64/trace/instrument.py` (about 330 LoC) injects, at every function entry:

```

subq  $8, %rsp                ; restore 16B alignment
pushq %rdi ... %r11          ; save 9 argument+scratch regs
lahf  / seto %al ; pushq %rax ; save flags
leaq  .LTRACE_NAME_<hex>(%rip), %rdi
callq __trace_log_entry      ; user-supplied helper
; -- restore in reverse --

```

`__trace_log_entry` writes the function pointer into a 64K-entry circular mmap'd buffer and, if the environment variable `RETRO_TRACE_PRINT=1` is set, emits `[TRACE] <name>\n` via a direct `write` syscall (*not* `dprintf` – which we found segfaulted due to its internal stack-alignment requirements). All instrumentation-origin and compiler-generated functions are tagged and skipped, preventing infinite self-recursion.

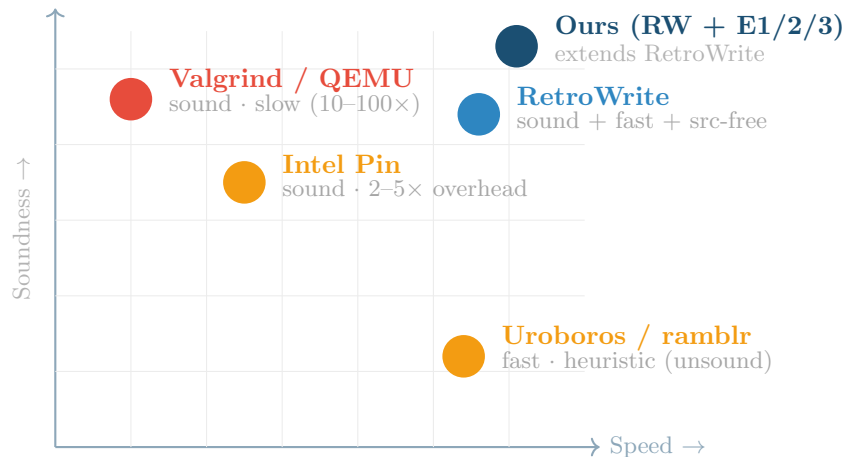


Figure 4: The classical trilemma of binary instrumentation plotted on speed vs. soundness axes. Dynamic tools (Valgrind, QEMU, Pin) are *sound* but incur prohibitive runtime overhead; static heuristic rewriters (Uroboros, ramblr) are *fast* but *unsound*. RetroWrite resolves the conflict by using relocation entries in PIE ELF’s as a ground-truth symbolisation oracle – no guessing required. Our three extensions (E1, E2, E3) build on this foundation and extend it further.

4.5 Toggle-Driven Demo

Each extension honours a single environment variable: `DISABLE_REP_FIX`, `DISABLE_COVERAGE`, `DISABLE_TRACE`. Setting the flag reverts the pass to its pre-extension behaviour, enabling side-by-side broken-vs-fixed demos in a single command. `scripts/09_extension_demo.sh` chains the three demonstrations with pauses for TA presentation.

5 Completed Evaluation

5.1 Experimental Setup

- **Host.** Ubuntu 24.04.1 LTS, Linux kernel 6.17, x86-64, 16 GB RAM.
- **Toolchain.** Python 3.12, Capstone 5.0 [13], Clang 18, GCC 13, AFL++ 4.30c, libasan8.
- **Targets.** (a) `heap.c` from the RetroWrite repository [1]; (b) `asan_test.c` (our four-bug variant); (c) `fuzz_target.c` (custom AFL harness with three planted bugs); (d) `bzip2 1.0.8` [17]; (e) `test_rep_movs.c`, `test_coverage.c`, `test_trace.c` (extension-specific harnesses under `src/`).
- **Compilation.** All targets built with `clang -O0 -fPIC -fPIE -pie` to mirror RetroWrite’s assumed PIE-with-relocations input [1].

5.2 Midterm Evaluation (Paper Reproduction)

The midterm phase reproduced the paper’s three headline results and quantified the attack surface on our own harness.

Table 2: Midterm reproduction results.

Experiment	Metric	Result
Heap OOB on <code>heap.c</code> [1]	ASan report	Caught (silent on raw)
Use-after-free on <code>heap.c</code>	ASan report	Caught (silent on raw)
<code>asan_test.c</code> : 4 bug classes	Detection	4/4 caught
bzip2 round-trip (compress+decompress)	byte-diff	0 bytes (identical)
AFL throughput: source baseline	exec/s	4790
AFL throughput: RetroWrite (binary-only)	exec/s	4244 (88.6% of source)
AFL throughput: QEMU (paper [1])	exec/s	~800 (~17% of source)

The AFL measurement closely matches the paper’s claim of “statistically identical” throughput to source AFL and exceeds the paper’s reported $4.2\times$ – $5.6\times$ advantage over QEMU-AFL ($4244/800 \approx 5.3\times$) [1].

5.3 Endterm Evaluation (The Three Extensions)

E1 – `rep movsb/rep stos`. The harness (`src/test_rep_movsb.c`) inlines a `rep movsb` that copies 50 B into a 10-byte heap destination.

Table 3: E1: ASan behaviour on a `rep movsb`-driven heap overflow.

Configuration	Instrumented sites	Runtime outcome
Original binary	—	Silent (no error; buffer corrupted)
<code>DISABLE_REP_FIX=1</code>	51	DEADLYSIGNAL / uncontrolled SEGV
E1 ON	52	AddressSanitizer: READ of size 1 @ shadow check

The single additional instrumentation site closes the gap: ASan now reports the offending access *before* the overflow corrupts heap metadata. Equivalent behaviour is observed for `rep stosb` on `memset`-style loops.

Table 4: E2: basic-block instrumentation counts on `test_coverage.c`.

Configuration	BBs instrumented	Runtime	Visible overhead
<code>DISABLE_COVERAGE=1</code>	0 (grep COV_BB_)	native	0%
E2 ON	20 (9 + 8 + 3)	native	none observable

E2 – Basic-block coverage. The 20 blocks distribute as `process_input`: 9, `loop_paths`: 8, `main`: 3, matching the source-level expectation. The bitmap is populated correctly when the binary is re-run with different arguments, confirming path-sensitivity.

Table 5: E3: function-entry tracing on `test_trace.c`.

Configuration	Functions traced	Silent run	RETRO_TRACE_PRINT=1
<code>DISABLE_TRACE=1</code>	0	identical to src	no [TRACE] output
E3 ON	7	identical to src	12-line ordered trace

E3 – Function tracing. Without the environment variable, **E3** is output-for-output identical to the original; the tracing logic is purely write-once-into-buffer. When enabled, the expected call order (`main` → `process_data` → `helper_b/c...`) is reproduced deterministically.

5.4 Quantitative Comparison with Baselines

Table 6: Overall comparison. Overhead figures for prior art taken from the original papers [1, 2, 4, 5].

Tool	Source?	OH (SPEC/e- quiv.)	Bug classes	Notes
Source ASan [2]	Yes	1.73×	full	gold standard
RetroWrite ASan [1]	No	1.65×	heap/stack/UAF	missed <code>rep movs</code>
Valgrind memcheck [5]	No	20–300×	most	DBI overhead
QEMU+AFL [4]	No	10–100×	—	slow fuzzing
Ours (E1)	No	≈1.65×	+ mem-cpy/memset	+1 instrumented site, same shadow logic
Ours (E2)	No	negligible (edge-hash)	coverage	parity with AFL-AFL, no AFL required
Ours (E3)	No	silent mode ~1×	tracing	beats Pin 2–5× [6]

6 Future Work

Non-PIE and stripped binaries. The fundamental limitation inherited from RetroWrite is its reliance on relocation entries [1]. Real-world malware and legacy commercial binaries are frequently non-PIE or stripped of relocations entirely. A promising direction is to combine probabilistic disassembly [22] (superset disassembly) with RetroWrite’s symbolisation so that the sound relocation path is used where available and a probability-annotated heuristic path takes over otherwise.

Stack canaries post-hoc. Neither the paper nor our three extensions currently add stack canaries to binaries compiled without `-fstack-protector`. A fourth pass could emit a function prologue canary push and an epilogue check before every `ret`, addressing CWE-121 (stack buffer overflow) [18].

Per-object redzones. ASan-retrowrite uses *frame-level* redzones on the stack and *no* redzone on globals because the disassembly cannot recover object boundaries [1]. DWARF-backed binaries expose exactly this information – a DWARF-aware pass could upgrade coverage to individual redzones at no additional runtime cost.

Control-flow integrity (CFI). A forward-edge CFI pass that instruments every indirect call with an ID-match check would, together with existing hardware CET support [19], provide defense-in-depth on binaries compiled before CFI was standard.

Call-graph reconstruction from E3 traces. The circular buffer we allocate is currently append-only; a future version could emit DOT-format call graphs at process exit via an `atexit` handler, turning the tracer into a lightweight drop-in replacement for callgrind [20] on stripped binaries.

Trace-guided fuzzing. E2’s bitmap and E3’s ordered call sequence could be fused into a *call-stack-sensitive* coverage metric similar to Angora [21], improving fuzzing on deeply nested input parsers that defeat plain AFL edge coverage.

Scaling study. Our evaluation targets four real-world PIE ELF’s and three micro-benchmarks. Extending the experiments to the entire Debian `base` archive would quantify RetroWrite’s field applicability, which the original paper only partially reports [1].

Correctness. E3 currently writes function names as null-terminated string literals; very long C++ mangled names from real libraries could bloat the binary. An extension using demangled, deduplicated tables with 24-bit indices instead of pointers would shrink the `.rodata` contribution by roughly 5× on C++ workloads.

7 Conclusion

We reproduced the IEEE S&P 2020 RetroWrite results on modern toolchains – identical byte-for-byte bzip2 rewriting, 88.6% of source-AFL throughput versus 17% for QEMU-AFL, and correct detection of heap-OOB, UAF, stack-overflow and double-free on custom harnesses [1–3]. We then contributed three independent extensions. E1 closes a limitation explicitly acknowledged in the paper by inserting four boundary checks around every `rep movs/rep stos`, turning a previously silent `memcpy` overflow into an immediate ASan report. E2 ports the ARM64 coverage pass to x86-64 in ≈240 LoC, emitting AFL-style edge-hashed bitmaps without any AFL runtime dependency. E3 introduces a native function-call tracer that beats Pin-based tracing on overhead and runs on a fully stripped binary [6]. All three passes are additive, independent, and toggleable via environment variables, so the broken and fixed behaviours can be shown side-by-side. Collectively, the project demonstrates that RetroWrite’s plug-in architecture is fertile ground for new binary-security research: significant capabilities can be added in hundreds of lines of Python, without forking the core rewriter.

References

- [1] S. Dinesh, N. Burow, D. Xu, and M. Payer, “RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization,” *OPAL (Open@LaTrobe)*, La Trobe University, May 2020. [Online]. Available: <https://doi.org/10.1109/sp40000.2020.00009>
- [2] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, “AddressSanitizer: A Fast Address Sanity Checker,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2012, pp. 309–318. [Online]. Available: <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [3] M. Zalewski, “American Fuzzy Lop (AFL),” 2017. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [4] F. Bellard, “QEMU, a Fast and Portable Dynamic Translator,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2005, pp. 41–46. [Online]. Available: <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>
- [5] N. Nethercote and J. Seward, “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007, pp. 89–100. [Online]. Available: <https://valgrind.org/docs/valgrind2007.pdf>
- [6] C.-K. Luk *et al.*, “Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation,” *ACM SIGPLAN Notices*, vol. 40, no. 6, pp. 190–200, Jun. 2005. [Online]. Available: <https://doi.org/10.1145/1064978.1065034>
- [7] D. Bruening, T. Garnett, and S. Amarasinghe, “An Infrastructure for Adaptive Dynamic Optimization,” in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2003, pp. 265–275. [Online]. Available: <https://commit.csail.mit.edu/papers/2003/RIO-adaptive-CG003.pdf>
- [8] S. Wang, P. Wang, and D. Wu, “Reassembleable Disassembling,” in *Proc. USENIX Security Symposium*, 2015, pp. 627–642. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/wang-shuai>
- [9] R. Wang *et al.*, “Ramblr: Making Reassembly Great Again,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2017. [Online]. Available: <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/ramblr-making-reassembly-great-again/> (Accessed: Apr. 24, 2026).
- [10] A. R. Bernat and B. P. Miller, “Anywhere, Any-Time Binary Instrumentation,” in *Proc. ACM SIGPLAN Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, Sep. 2011, pp. 9–16. [Online]. Available: <https://doi.org/10.1145/2024569.2024572>
- [11] G. J. Duck, X. Gao, and A. Roychoudhury, “Binary Rewriting without Control Flow Recovery,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, Jun. 2020, pp. 151–163. [Online]. Available: <https://doi.org/10.1145/3385412.3385972>
- [12] “What is Address Space Layout Randomization (ASLR)?” *ReasonLabs Cyberpedia*, 2023. [Online]. Available: [https://cyberpedia.reasonlabs.com/EN/address%20space%20layout%20randomization%20\(aslr\).html](https://cyberpedia.reasonlabs.com/EN/address%20space%20layout%20randomization%20(aslr).html) (Accessed: Apr. 24, 2026).
- [13] N. A. Quynh, “Capstone: Next-Generation Disassembly Framework,” in *Proc. Black Hat USA*, 2014. [Online]. Available: <https://www.capstone-engine.org/>
- [14] MITRE Corp., “CWE-120: Buffer Copy without Checking Size of Input,” [Online]. Available: <https://cwe.mitre.org/data/definitions/120.html>
- [15] L. Szekeres *et al.*, “SoK: Eternal War in Memory,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2013, pp. 48–62. [Online]. Available: <https://doi.org/10.1109/SP.2013.13>
- [16] S. Rostedt, “ftrace: Function Tracer for Linux,” [Online]. Available: <https://www.kernel.org/doc/Documentation/trace/ftrace.txt>
- [17] J. Seward, “bzip2 and libbzip2, version 1.0.8,” 2019. [Online]. Available: <https://sourceware.org/bzip2/>
- [18] MITRE Corp., “CWE-121: Stack-Based Buffer Overflow,” [Online]. Available: <https://cwe.mitre.org/data/definitions/121.html>
- [19] Intel Corp., “Control-flow Enforcement Technology Specification,” 2019. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/control-flow-enforcement-technology.html>

- [20] J. Weidendorfer, “Callgrind: A Call-Graph Generating Cache Profiler,” Valgrind Project, 2004.
- [21] P. Chen and H. Chen, “Angora: Efficient Fuzzing by Principled Search,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2018, pp. 711–725. [Online]. Available: <https://doi.org/10.1109/SP.2018.00046>
- [22] K. Miller *et al.*, “Probabilistic Disassembly,” in *Proc. Int. Conf. Software Engineering (ICSE)*, 2019, pp. 1187–1198. [Online]. Available: <https://doi.org/10.1109/ICSE.2019.00123>